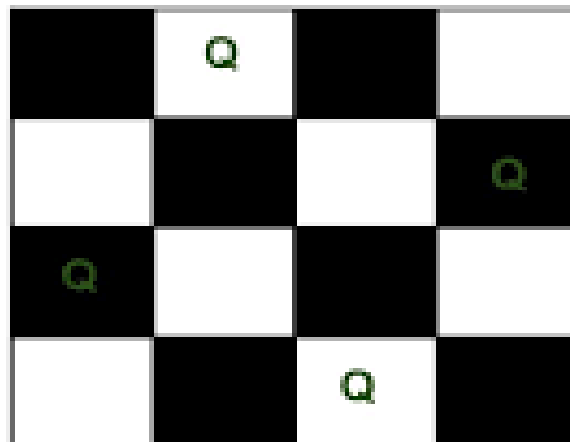




N Queens

- The N Queen is the problem of placing N chess queens on an $N \times N$ chessboard so that no two queens attack each other.
- **Example**
 - The following is a solution for 4 Queen problem. The expected output is a binary matrix which has 1s for the blocks where queens are placed.



N Queens Problem

What is Backtracking?

- In backtracking, we start with one possible move out of many available moves. We then try to solve the problem.

What is the N-Queens Problem?

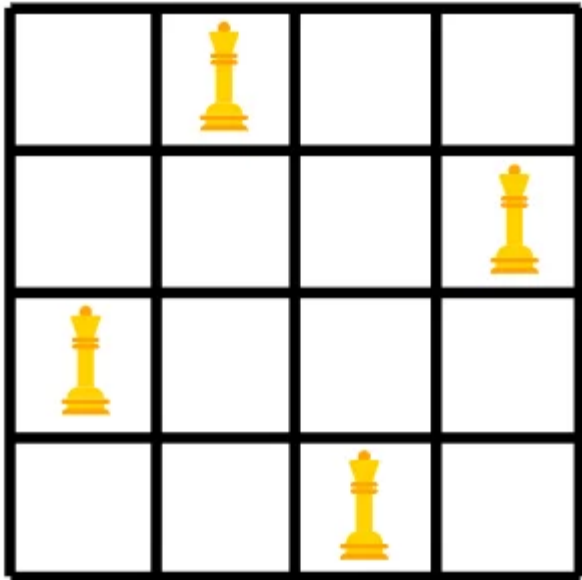
- How can N queens be placed on an NxN chessboard so that no two of them attack each other?
- This problem is commonly seen for N=4 and N=8.
- Let's look at an example where **N=4**

N Queens Problem-Movement of Queen

- **Before solving the problem**, you need to know about the movement of the queen in chess.
- A queen can move any number of steps in any direction. The only constraint is that it can't change its direction while it's moving.
- One thing that is clear by looking at the queen's movement is that no two queens can be in the same row or column.
- That allows us to place only one queen in each row and each column.
- When **N=4**, the solution looks like :

N Queens Problem-Movement of Queen

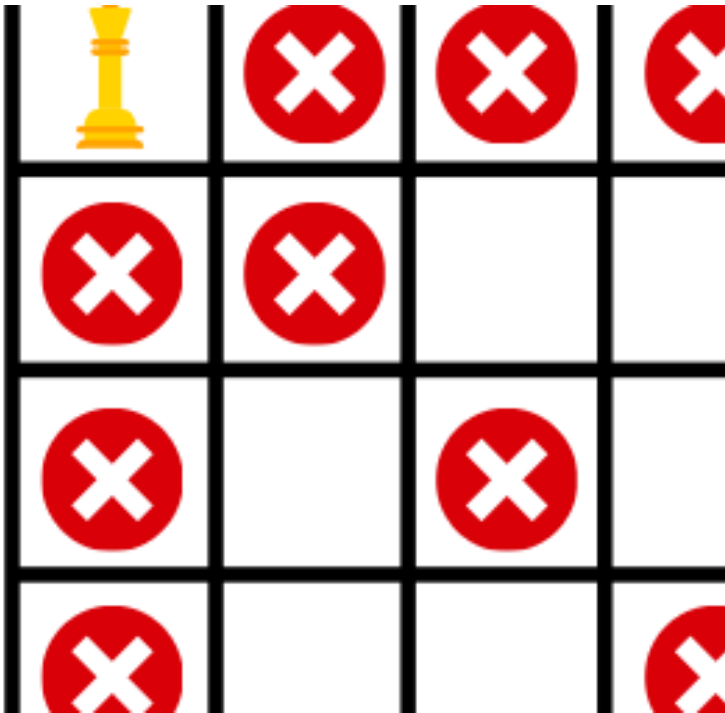
Queen solution



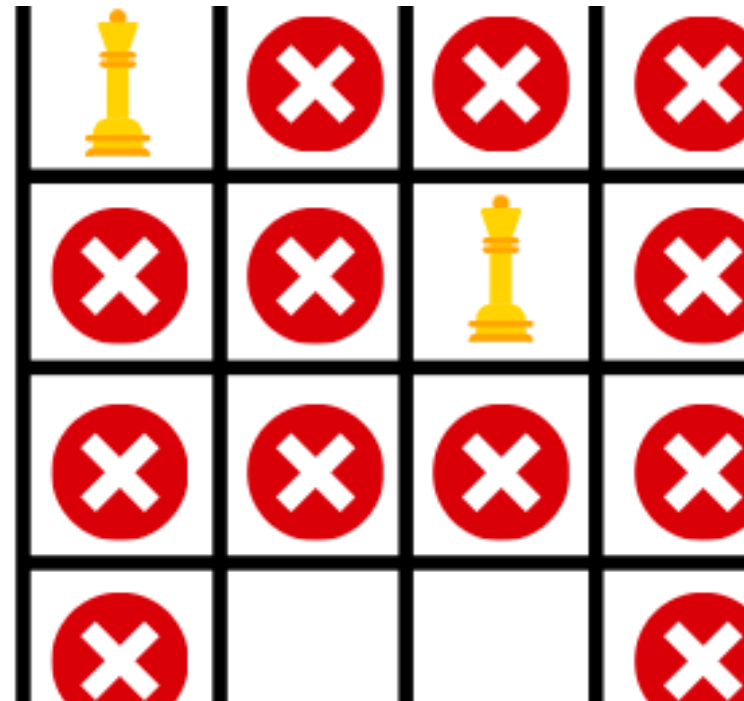
- **Solution to the N-Queens Problem**
- The way we try to solve this is by placing a queen at a position and trying to rule out the possibility of it being under attack. We place one queen in each row/column.
- If we see that the queen is under attack at its chosen position, we try the next position.
- If a queen is under attack at all the positions in a row, we backtrack and change the position of the queen placed prior to the current position.
- We repeat this process of placing a queen and backtracking until all the N queens are placed successfully.

The step by step backtracking is shown as follows:

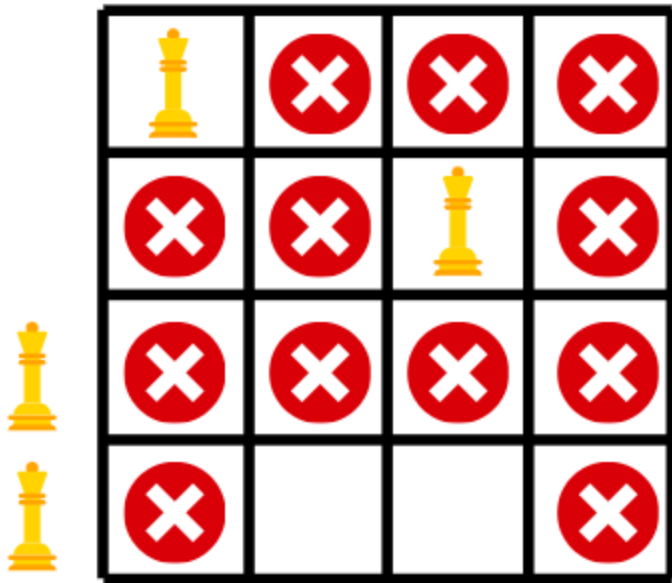
Start



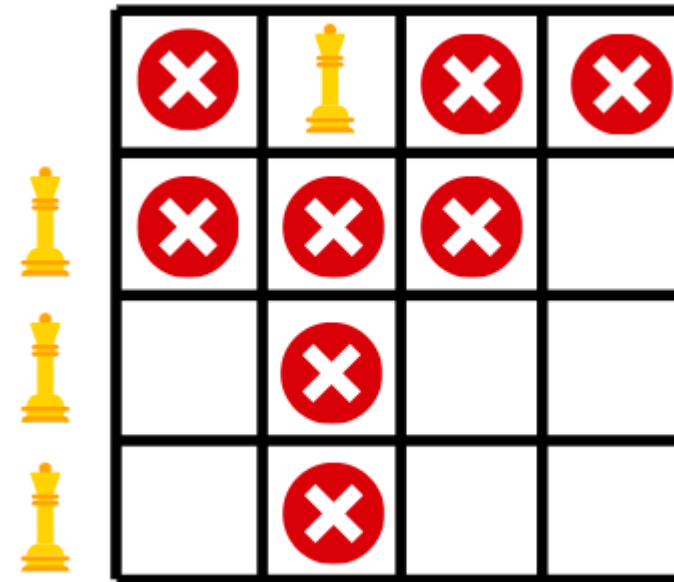
No place for queen 3, so
we backtrack



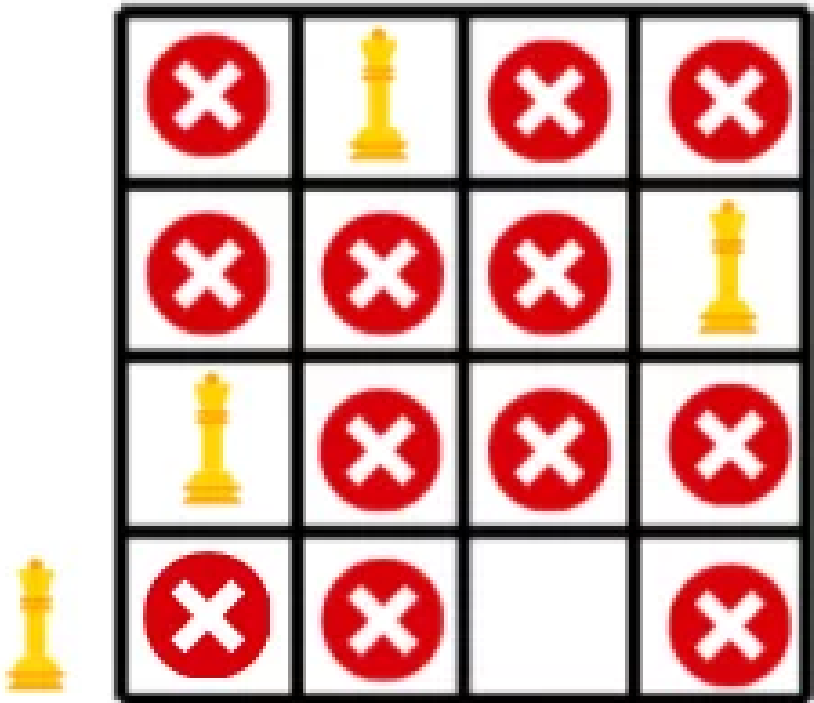
After backtracking we are not able to place queen 4, so again we backtrack.



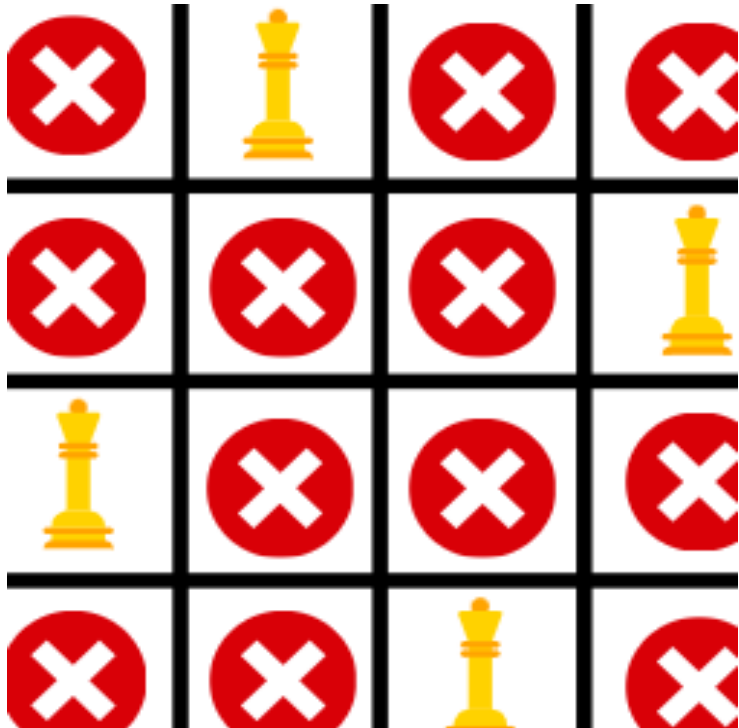
This time we backtrack all the way to the first queen.



First three queens placed successfully after backtracking.



All the queens placed




```

1. public class NQueenProblem {
2.     final int N = 4;
3.     void printSolution(int board[][]){
4.         for (int i = 0; i < N; i++) {
5.             for (int j = 0; j < N; j++)
6.                 System.out.print(" " +
board[i][j]
7.                     + " ");
8.             System.out.println();    }
9.     }
10.    boolean isSafe(int board[][], int row,
int col){
11.        int i, j;
12.        for (i = 0; i < col; i++)
13.            if (board[row][i] == 1)
14.                return false;

15.        for (i = row, j = col; i >= 0 && j >=
0; i--, j--)

```

```

1.     if (board[i][j] == 1)
2.         return false;

3.         for (i = row, j = col; j >= 0 && i < N; i++, j--)
4.             if (board[i][j] == 1)
5.                 return false;
6.         return true;
7.     }
8.    boolean solveNQUtil(int board[][], int col){

9.        if (col >= N)
10.            return true;

11.        for (int i = 0; i < N; i++) {
12.            if (isSafe(board, i, col)) {

13.                board[i][col] = 1;

```

```

1.     if (solveNQUtil(board, col + 1) == true)
2.         return true;
3.         board[i][col] = 0; // BACKTRACK
4.     }    }
5.     return false;
6. }
7. boolean solveNQ()
8. {
9.     int board[][] = { { 0, 0, 0, 0 },
10.        { 0, 0, 0, 0 },
11.        { 0, 0, 0, 0 },
12.        { 0, 0, 0, 0 } };
13.     if (solveNQUtil(board, 0) == false) {
14.         System.out.print("Solution does not exist");
15.         return false;
16.     }
17.     printSolution(board);
18.     return true;
19. }

```

```

1.     public static void main(String args[]){
2.         NQueenProblem Queen = new NQueenProblem();
3.         Queen.solveNQ();
4.     }}

```

Output:

```

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

```



THANK YOU